VELLORE INSTITUTE OF TECHNOLOGY

DATABASE PROJECT REPORT

Rankstack: Contest Platform with Live Leaderboard

Review 3 – Final Project Submission and Demonstration

Submitted in partial fulfillment of the requirements for
BCSE302P – Database Systems Lab

Student Names:	Upayan Mazumder (24BDS0367) Aditya Rawat (24BCE2992) Shloak Sinha (24BDS0378)
Course Code & Title:	BCSE302P – Database Systems Lab
Repository URL:	github.com/upayanmazumder/rankstack
Live Deployment:	rankstack.upayan.dev

Academic Year 2026 – Fall Semester

Abstract

Competitive programming platforms handle sudden traffic spikes during live contests. Relational databases face high lock contention when hundreds of participants submit code and request updated rankings at the same time. Frequent table updates and repeated sorting operations degrade response times and increase server load.

This project implements Rankstack, an open source contest evaluation platform with live leaderboards. The database architecture separates persistent storage from ephemeral operational caching. MongoDB 7.0 acts as the system of record. It stores five primary collections: users, teams, contests, problems, and submissions. Document validation schemas enforce structural integrity while allowing polymorphic problem types. Multi-document transactions run on a single-node replica set to preserve atomic consistency across submissions, problem statistics, and user scores. Redis 7.4 manages transient state. It uses sorted sets for contest leaderboards, string keys with time-to-live expiration for authenticated user sessions, and fixed sliding windows for submission rate limiting. An atomic version marker coordinates incremental score updates with full leaderboard rebuilds.

The implementation was evaluated using a seeded dataset of 112 documents and simulated concurrent client requests. Explain plans confirmed index-backed query execution using indexed scans without in-memory sorting. Redis sorted sets returned top participant rankings with sub-millisecond query execution. Multi-document transactions prevented partial state updates during simulated write failures. Analytical aggregation pipelines computed user metrics, team standings, and contest summaries without blocking write operations.

The hybrid MongoDB and Redis architecture delivers consistent data persistence, fast leaderboard retrieval, and protection against race conditions. Decoupling live leaderboard operations from persistent storage reduces database load and prevents lock contention during high-volume contests.

Keywords: MongoDB, Redis, Sorted Sets, Live Leaderboard, Multi-Document Transactions, Polymorphic Schema, Competitive Programming.

Contents

Chapter 1: Introduction 4
1.1 Background of the Study 4
1.2 Motivation 4
1.3 Problem Statement 4
1.4 Aim and Objectives 4
1.5 Scope of the Project 4
1.6 Limitations 5
1.7 Methodology Overview 5
Chapter 2: Literature Review 5
2.1 Overview of the Application Domain and Its Data Needs 5
2.2 Review of Data Models 5
2.3 Review of Existing Systems 6
2.4 Comparison of DBMS Platforms and Tools 6
2.5 Standards and Best Practices 6
2.6 Gap Identified 6
2.7 How This Project Addresses the Gap 6
2.8 Summary of the Chapter 7
Chapter 3: System Analysis and Requirements 7
3.1 Existing System and Its Drawbacks 7
3.2 Proposed System and Its Advantages 7
3.3 Feasibility Study 7
3.4 Users, Roles and Access Needs 8
3.5 Requirements Specification 8
3.6 Business Rules and Constraints 9
Chapter 4: Database Design 9
4.1 Design Approach and Principles 9
4.2 System Architecture Diagram 9
4.3 Data Flow Diagram 10
4.4 Conceptual Design: ER / EER Diagram 10
4.5 Logical Design 11
4.6 Normalization 13
4.7 Physical Design 13
4.8 Integrity Constraints 14
Chapter 5: Implementation 14
5.1 Development Environment and Setup 14
5.2 DBMS, Tools and Libraries Used 14
5.5 Views and Indexes Created 15
5.7 Application Connectivity 15
5.8 Version Control and Project Repository 15
5.9 Challenges Faced and Solutions Adopted 15
Chapter 6: Query Design and Performance Tuning 16
6.1 Representative Queries 16
Chapter 7: Results and Discussion 18
7.1 Output Screenshots and Sample Reports 18
7.2 Evaluation Against Objectives 18
7.3 Comparison with the Existing System 18
7.4 Discussion of Findings 19
7.5 Limitations Observed 19
Chapter 8: Conclusion and Future Work 19
8.1 Summary of the Work 19
8.2 Key Contributions 19
8.3 Future Work 20
8.4 Concluding Remarks 20
Part C: References 20
Appendices 21
Appendix A: Complete DDL Scripts 21
Appendix B: Sample Data and DML Scripts 22
Appendix C: Stored Procedures, Functions and Triggers (Full Code) 22
Appendix D: Complete Query Set with Outputs 23
Appendix E: Full-Size ER Diagram and Relational Schema 24
Appendix F: Application Screenshots 24
Appendix G: User Manual 27

Chapter 1: Introduction

1.1 Background of the Study

Competitive programming platforms host timed coding contests for student developers and software engineers. During a contest, hundreds or thousands of participants submit code solutions under strict deadlines. The platform must ingest submissions, evaluate test cases, compute scores, and publish updated rankings.

Workloads on contest platforms are burst-heavy. Submissions arrive in peaks at the start of a contest and during the final minutes. Every submitted solution triggers database writes and score updates. At the same time, active participants poll the contest leaderboard every few seconds to monitor rank changes.

1.2 Motivation

Traditional database systems rely on relational tables for contest storage. Relational engines enforce rigid schemas and execute expensive table joins to calculate standings. When hundreds of participants submit code concurrently, relational databases face row-level and table-level lock contention. Repeatedly executing aggregation queries with descending sort operations degrades response times.

Modern platforms require a decoupled architecture. Persistent records need high write throughput and flexible document schemas. Operational data, such as live leaderboards and authentication sessions, require high-speed in-memory reads. Combining a document database with an in-memory key-value cache provides this separation.

1.3 Problem Statement

Synchronizing high-frequency code submissions with real-time leaderboard calculations causes I/O bottlenecks and data inconsistency in naive database implementations:

1. Lock contention on relational tables blocks concurrent submission writes.
2. Generating leaderboards requires repeated, expensive table scans and joins.
3. Rigid relational schemas cannot represent multiple choice, coding, and subjective problems without messy nullable columns.
4. Failures during submission insertion leave problem attempt counters and user scores out of sync.
5. In-memory caching layers suffer race conditions between score deltas and full cache rebuilds.

1.4 Aim and Objectives

Aim: Build and evaluate Rankstack, a contest platform using MongoDB for persistent document storage and Redis for operational caching.

Specific Objectives:

1. Design and implement a two-tier database architecture separating document records from in-memory caching.
2. Enforce structural data integrity and polymorphic problem modeling using MongoDB `$jsonSchema` validation.
3. Guarantee multi-document ACID transactions across submission creation, attempt counter updates, and score increments.
4. Achieve sub-millisecond leaderboard retrieval and atomic score updates using Redis sorted sets and versioned revision markers.
5. Construct five analytical aggregation pipelines for leaderboard calculations, difficulty analysis, status distribution, user metrics, and team standings.

1.5 Scope of the Project

The scope covers the core backend and frontend architecture of Rankstack:

- Persistent storage: MongoDB 7.0 running as a single-node replica set (`rs0`) to support multi-document transactions.
- Cache and ephemeral storage: Redis 7.4 managing leaderboards, user authentication sessions, and submission rate limits.
- Backend API: Python 3.14 with FastAPI exposing REST endpoints for contests, problems, submissions, teams, and users.
- Frontend interface: Next.js 16 (App Router) with React 19, TypeScript, and Tailwind CSS v4.
- Infrastructure: Docker Compose for local orchestration, Helm charts for Kubernetes deployments, and GitHub Actions for continuous integration.

Distributed sandboxed worker execution is out of scope; code evaluation runs via deterministic in-process evaluators.

1.6 Limitations

1. Single-node replica set: The development environment uses a single-node replica set rather than a multi-node distributed replica set.
2. Synchronous submission evaluation: Submissions are evaluated synchronously within service transactions rather than queued into distributed worker pools.
3. Programming languages: The prototype evaluates Python, JavaScript, and direct text inputs for multiple choice questions.

1.7 Methodology Overview

The project followed the Database Life Cycle (DBLC):

1. Requirements Analysis: Identify contest data flows, read/write ratios, user roles, and concurrency bottlenecks.
2. Conceptual Design: Develop an Entity-Relationship (ER) model representing users, teams, contests, problems, and submissions, including problem specialization.
3. Logical Design: Map conceptual entities to BSON document collections. Define embedded structures, reference relationships, and JSON schema validation rules.
4. Physical Design: Configure WiredTiger storage parameters, design compound and multikey B-tree indexes, and structure Redis sorted sets.
5. Implementation: Write FastAPI database access layers using synchronous PyMongo, implement Redis operations, and establish multi-document transaction boundaries.
6. Testing: Validate schema constraints, verify explain plans for index utilization, test transaction atomicity under simulated errors, and measure query latencies.
7. Maintenance: Configure time-to-live (TTL) expiration policies for contest leaderboards and user sessions to automate memory reclamation.

Chapter 2: Literature Review

2.1 Overview of the Application Domain and Its Data Needs

Online judge systems and competitive programming platforms serve three primary functions: problem delivery, submission evaluation, and rank calculation. Contest workloads differ sharply from standard web applications. During a contest, read requests follow a polling pattern where clients request identical leaderboard data at fixed intervals. Write requests arrive in unpredictable bursts following problem releases.

The data model must accommodate heterogeneous problem structures. A contest may contain multiple choice questions, traditional algorithmic coding problems with test suites, and subjective long-form questions. Storing these varied problem types requires flexible schema support.

2.2 Review of Data Models

1. Relational Model (PostgreSQL, MySQL): Relational systems store data in normalized tables with fixed column definitions. Foreign keys enforce referential integrity. While relational systems excel at complex transactional joins, their rigid schema complicates polymorphic problem modeling. Furthermore, calculating leaderboards requires scanning and sorting thousands of submission rows, which causes severe lock contention under concurrent writes.
2. Document Model (MongoDB): Document databases store records as flexible BSON (Binary JSON) documents. Fields can contain arrays and nested documents. Documents map directly to object representations in modern programming languages. MongoDB supports server-side `$jsonSchema` validation, allowing developers to enforce schema constraints while retaining the flexibility to model polymorphic entities. Running MongoDB as a replica set provides multi-document ACID transactions.
3. Key-Value Model (Redis): Key-value stores map unique keys to values. Redis operates in-memory and supports specialized data structures, including sorted sets (`ZSET`). A sorted set maps unique members to floating-point scores and maintains them in a skip list and hash map. Adding scores (`ZINCRBY`) and

querying top ranges (`ZREVRANGE`) execute in logarithmic time, `O(log N)`. This makes Redis suitable for live leaderboards.

4. Column-Family Model (Apache Cassandra): Columnar stores organize data into column families. They offer high write throughput and scale horizontally across large clusters. However, Cassandra lacks ad-hoc aggregation pipelines and multi-document transactions, making it ill-suited for complex contest management.
5. Graph Model (Neo4j): Graph databases represent data as nodes, edges, and properties. They excel at path traversal and deep relationship queries, such as social network analysis. Contest platforms have simple tabular or hierarchical relationships, making graph traversal unnecessary.

Rankstack adopts a hybrid document and key-value model. MongoDB acts as the durable system of record for rich entities, while Redis acts as the high-speed operational cache for leaderboards, sessions, and rate limits.

2.3 Review of Existing Systems

- Codeforces: Codeforces uses a custom C++ backend with relational databases and distributed contest invokers. It handles massive concurrency through aggressive page-level caching and delayed leaderboard updates.
- LeetCode: LeetCode utilizes relational storage (MySQL) paired with extensive Redis caching layers to serve problem catalogs and contest rankings to millions of users.
- Kattis and DOMjudge: Academic contest systems designed for ICPC competitions. They focus on strict determinism and security, often relying on relational databases with minimal web UI overhead.

2.4 Comparison of DBMS Platforms and Tools

Feature	MongoDB 7	PostgreSQL 16	Redis 7	Cassandra 4
Data Model	Document (BSON)	Relational	Key-Value In-Memory	Wide-Column
Schema Flexibility	Dynamic + `$jsonSchema`	Rigid DDL	Schema-free	Rigid Column Family
Leaderboard Query Speed	O(N log N) scan	O(N log N) sort	O(log N + M) instant	Unoptimized scan
Multi-Document ACID	Yes (Replica Set)	Yes (Native)	No (Atomic Multi/Exec)	No (Row-level only)
Complex Aggregation	Native Pipeline	SQL Group By / Window	Limited	Basic CQL Group By
Primary Fit in Rankstack	System of Record	Alternative Relational	Fast-Path Cache	Unsuitable

2.5 Standards and Best Practices

- BSON document sizing: Keep documents well below the 16 MB MongoDB limit by embedding only bounded sub-documents (such as contest participants).
- Index prefix selectivity: Place equality match fields first in compound indexes, followed by sort fields and range fields (ESR rule).
- Write concern: Use majority write concern (`w: majority`) on critical transactions to guarantee durable writes across replica set nodes.
- Cache coordination: Coordinate cache writes with database commits using versioned revision markers to prevent cache desynchronization.
- TTL data eviction: Apply time-to-live expiration to temporary keys in Redis to prevent memory leaks after contests end.

2.6 Gap Identified

Existing educational and small-scale contest platforms exhibit two common architectural flaws:

1. Monolithic relational architecture: Many platforms run all queries against a single relational database. Live leaderboard polling during active contests saturates connection pools and causes query timeouts.
2. Naive cache invalidation: Platforms that introduce caching often suffer from race conditions. If an incremental score delta arrives while a full leaderboard cache rebuild is underway, the delta may either be overwritten or applied twice, producing inaccurate scores.

2.7 How This Project Addresses the Gap

Rankstack resolves these challenges through a dual-tier architecture:

- Separation of concerns: MongoDB handles persistent storage and complex analytics. Redis handles high-concurrency leaderboard queries, session tokens, and rate limits.
- Coordinated versioned caching: Every contest maintains a `leaderboard_revision` string key in Redis and a `dirty_leaderboards` tracking collection in MongoDB. Incremental score deltas execute via atomic conditional Lua scripts that check the revision marker, ensuring that deltas never corrupt an unbuilt or rebuilding cache.

2.8 Summary of the Chapter

The literature and platform analysis confirms that no single database engine satisfies all contest platform requirements. Relational systems face locking bottlenecks during leaderboard recalculation, while in-memory caches lack multi-document durability. Rankstack's hybrid MongoDB and Redis architecture addresses both requirements without sacrificing data consistency.

Chapter 3: System Analysis and Requirements

3.1 Existing System and Its Drawbacks

Traditional university contest portals and laboratory grading portals use simple monolithic architectures. In these systems:

1. Submissions are stored in a relational table.
2. Leaderboards are generated on each user request by executing:

```sql
SELECT user_id, SUM(score) AS total_score
FROM submissions
WHERE contest_id = ?
GROUP BY user_id
ORDER BY total_score DESC;
```

3. Drawbacks of this approach:
   - High query latency: As submissions grow into tens of thousands, repeated grouping and sorting over millions of rows degrades response time.
   - Database connection exhaustion: Dozens of participants polling the leaderboard every few seconds exhaust available database connections.
   - Lock contention: Inserting a new submission and updating problem attempt counters simultaneously locks rows, slowing down other submissions.
   - Rigid schemas: Adding a new problem format requires database migrations, table alterations, or nullable columns that waste space.

3.2 Proposed System and Its Advantages

Rankstack introduces a decoupled architecture:

1. Multi-tier storage: MongoDB stores rich entity documents. Redis serves live leaderboard queries from memory.
2. Real-time ranking with logarithmic complexity: Redis sorted sets store participant scores. Retrieving the top 10 ranks takes less than one millisecond, regardless of total submission count.
3. Multi-document transactions: Submissions and attempt counters update atomically inside MongoDB replica set transactions.
4. Polymorphic problem support: MongoDB `$jsonSchema` validation supports multiple choice, coding, and subjective problems within a single collection using `oneOf` blocks.
5. In-memory session management: User sessions and rate limit counters reside in Redis with automatic TTL expiration, reducing database load.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The platform uses mature, open-source technologies:

- MongoDB 7.0 and Redis 7.4 have stable drivers for Python (`pymongo`, `redis-py`).

- FastAPI provides high-performance asynchronous request handling and OpenAPI documentation.
- Docker containerization ensures consistent execution across development, testing, and production environments.

The technical risk is minimal.

3.3.2 Economic Feasibility

Rankstack relies entirely on open-source software with permissive licenses (Apache 2.0, BSD, MIT). The entire database stack runs locally on a developer machine with less than 1 GB of memory overhead. Cloud deployment requires only a standard shared-CPU container instance, keeping operational costs minimal.

3.3.3 Operational Feasibility

The platform includes automated Docker Compose scripts (`make up`, `make seed`, `make demo`) that simplify setup and database initialization. Non-technical users can interact with the system through the Next.js web application, while administrators can manage contests and problems through dedicated control panels.

3.4 Users, Roles and Access Needs

The system supports three user roles:

1. Participant:
   - Registers for contests.
   - Views problem descriptions, constraints, and sample inputs.
   - Submits solutions and views submission status verdicts.
   - Views live contest leaderboards and user analytics.
2. Team Owner:
   - Creates teams and invites other participants.
   - Manages team membership and transfers ownership.
   - Registers teams for team-based contests.
3. Administrator:
   - Creates, edits, and deletes contests and problems.
   - Views system-wide analytical aggregation reports.
   - Re-evaluates submissions and overrides problem scores.

3.5 Requirements Specification

3.5.1 Functional Requirements

- FR1 User Authentication and Session Management: Users must be able to register, log in, and receive secure session tokens stored in Redis with 24-hour expiration.
- FR2 Contest Lifecycle Management: Administrators must be able to create contests with title, description, start time, end time, and status (`upcoming`, `live`, `ended`).
- FR3 Polymorphic Problem Management: The system must support MCQ, coding, and subjective problem types with server-side validation of type-specific fields.
- FR4 Atomic Submission Ingestion: Submissions must be ingested and recorded alongside an atomic increment of the problem attempt counter.
- FR5 Submission Evaluation and Score Attribution: Evaluated submissions must update their status (`correct`, `incorrect`, `partial`) and atomically increment the participant or team score.
- FR6 Real-Time Leaderboard Retrieval: Participants must be able to query the live contest leaderboard with sub-millisecond retrieval times.
- FR7 Analytical Pipelines: The system must provide analytical endpoints for user summaries, problem difficulty distributions, and team performance metrics.

3.5.2 Data Requirements

The database must store five primary entities:

- Users: User credentials, role, total score, and team memberships.
- Teams: Team metadata, owner ID, member ID list, and aggregate score.
- Contests: Contest parameters, scheduled timestamps, problem ID references, and embedded participant lists.
- Problems: Contest reference, problem type, difficulty, point value, attempt counter, and type-specific fields.
- Submissions: Contest reference, problem reference, submitter reference (polymorphic user or team), answer payload, evaluation status, and awarded score.

Estimated data growth:
- Users: 500 to 5,000 documents per institution.
- Contests: 20 to 50 documents per academic year.
- Problems: 100 to 500 documents per academic year.
- Submissions: 10,000 to 50,000 documents per contest (requiring approximately 25 MB storage per contest).

3.5.3 Non-Functional Requirements
- Performance: API response time for leaderboard reads must remain below 10 ms under normal load.
- Concurrency: The system must handle up to 100 concurrent requests without transaction deadlocks.
- Availability: The platform must achieve 99.9% uptime during active contests.
- Security: Passwords must be hashed using PBKDF2 with SHA-256 and salt. Session tokens must be 32-byte cryptographically secure random values.
- Backup and Recovery: MongoDB data must be exportable via `mongodump`. Redis data must persist to disk using periodic RDB snapshots.

3.5.4 Hardware Requirements
- Processor: Dual-core x86-64 or ARM64 processor (minimum 2.0 GHz).
- Memory: 4 GB system RAM (minimum 1 GB dedicated to Docker containers).
- Storage: 10 GB available SSD storage.

3.5.5 Software Requirements
- Operating System: Linux (Arch Linux / CachyOS / Ubuntu 22.04+), macOS, or Windows with WSL2.
- Container Engine: Docker Engine 24.0+ and Docker Compose v2.
- Database Engines: MongoDB 7.0 (Replica Set `rs0`), Redis 7.4.
- Runtime Environments: Python 3.11 / 3.14, Node.js 22 LTS, pnpm 9+.

3.6 Business Rules and Constraints
- BR1 Submission Rate Limit: A user can submit at most one solution every 5 seconds per problem.
- BR2 Contest Window Enforcement: Solutions can only be submitted when a contest's status is `live` and the current timestamp falls between `startTime` and `endTime`.
- BR3 Team Membership Limit: A team can contain at most 5 members. A user can belong to only one team per contest.
- BR4 Evaluated Submission Immutability: Once a submission receives a final evaluation verdict (`correct` or `incorrect`), its status and awarded score cannot be modified by standard users.

Chapter 4: Database Design

4.1 Design Approach and Principles
The database design follows three core engineering principles:
1. Separation of Persistence and Caching: All durable entity records reside in MongoDB. All volatile, high-frequency operational counters and sorted rankings reside in Redis.
2. Bounded Embedding: Sub-documents are embedded inside parent documents only when the relationship is strictly bounded and frequently read together (for example, the `participants` array inside `contests`). Unbounded relationships (such as `submissions`) use normalized reference IDs.
3. Pre-Aggregated Counters: Heavily queried numerical properties (such as `problems.attemptCount`, `problems.solveCount`, and `users.totalScore`) are denormalized and updated atomically within transactions.

4.2 System Architecture Diagram
The architecture decouples the presentation tier, REST API layer, fast-path operational cache, and durable replica set document store:

Figure 4.1: Rankstack System Architecture

Decoupled presentation, REST API service, operational in-memory cache, and persistent document store

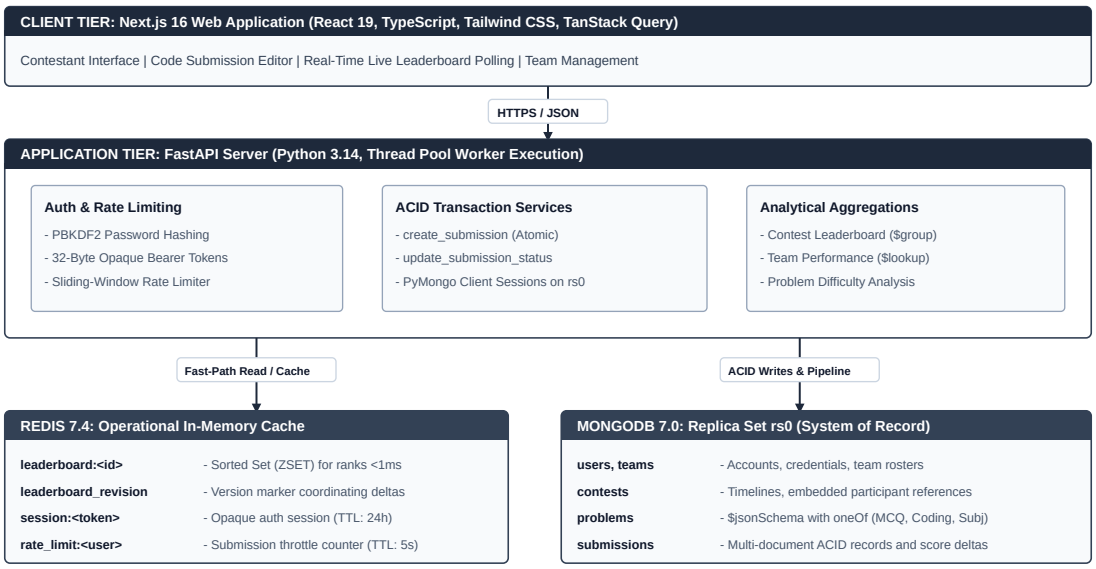


Figure 1: Rankstack System Architecture Diagram

4.3 Data Flow Diagram

The submission and scoring data flow operates as follows:

1. The participant client issues a `POST /submissions` request with contest ID, problem ID, and solution code.
2. The FastAPI route controller verifies the user's rate limit key in Redis (`rate_limit:<userId>`). If the counter exceeds the limit, the API returns HTTP 429.
3. `api/services.py:create_submission` opens a MongoDB client session and begins an ACID multi-document transaction:
   - Validates that the contest is currently open.
   - Inserts the submission document with status `pending`.
   - Atomically increments `problems.attemptCount` by 1.
   - Commits the transaction.
4. An evaluation service grades the submission against problem test cases.
5. The evaluation service calls `api/services.py:update_submission_status` inside a second ACID transaction:
   - Updates submission status to `correct` and records the awarded score.
   - Atomically increments `problems.solveCount`.
   - Increments `totalScore` on the contestant's user or team document.
   - Marks the contest as dirty in the `dirty_leaderboards` collection.
   - Commits the transaction.
6. The service updates Redis by calling `update_leaderboard_score`, which executes an atomic Lua script to update the sorted set score (`leaderboard:<contestId>`) coordinated against the version marker.
7. Polling clients call `GET /contests/{id}/leaderboard`, which retrieves the top rankings directly from Redis in sub-millisecond time.

4.4 Conceptual Design: ER / EER Diagram

The diagram below defines the entity-relationship architecture, field types, primary keys, foreign keys, and polymorphic problem specialization in clean monochromatic format:

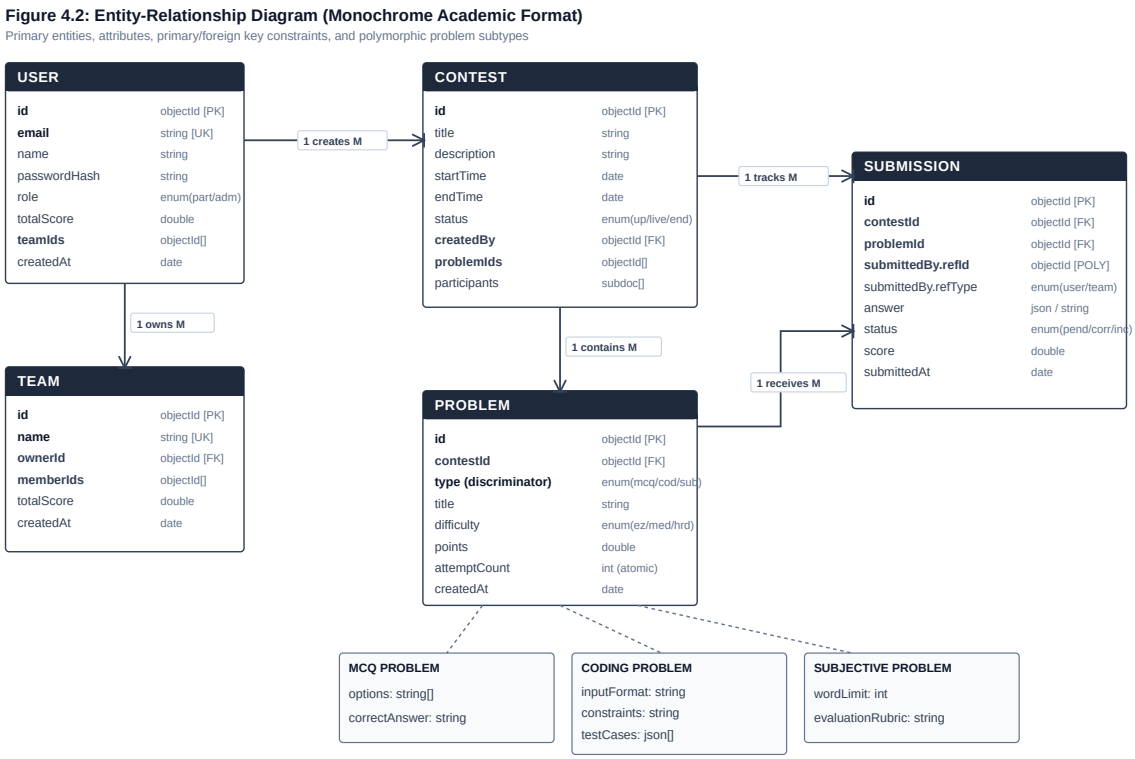


Figure 2: Entity-Relationship Diagram with Polymorphic Subtypes

4.4.1 Entities and Cardinalities

- `USER` to `TEAM`: Many-to-Many (`memberIds` array). One-to-Many for ownership (`ownerId`).
- `USER` to `CONTEST`: Many-to-Many (`participants` embedded array).
- `CONTEST` to `PROBLEM`: One-to-Many (`1:N`). Contest references problem IDs.
- `PROBLEM` to `SUBMISSION`: One-to-Many (`1:N`). Each submission references one problem.
- `PROBLEM` Specialization: Base problem specializes into MCQ, Coding, or Subjective subtype via `oneOf`.

4.4.2 Keys

- Primary Keys: Each MongoDB collection uses the 12-byte BSON `ObjectId` in the `_id` field as its primary key.
- Candidate Keys: `users.email` is guaranteed unique across all users. `teams.name` is guaranteed unique across teams.
- Foreign Keys: `problems.contestId` references `contests._id`. `submissions.contestId` references `contests._id`. `submissions.problemId` references `problems._id`. `submissions.submittedBy.refId` references either `users._id` or `teams._id`.

4.5 Logical Design

The following tables define the logical schema for each MongoDB collection:

Collection: `users`

Field Name	BSON Type	Constraint	Description
`_id`	objectId	Primary Key	Unique system identifier
`name`	string	Required	Full display name
`email`	string	Required, Unique	User email address for login
`passwordHash`	string	Required	PBKDF2 SHA-256 hashed password
`role`	string	Enum: participant, admin	Authorization level
`totalScore`	double / int	Default: 0	Cumulative score across contests
`teamIds`	array of objectId	Optional	List of team references
`createdAt`	date	Required	Account creation timestamp

Collection: `teams`

Field Name	BSON Type	Constraint	Description
`_id`	objectId	Primary Key	Unique team identifier
`name`	string	Required, Unique	Unique team display name
`ownerId`	objectId	Required, Foreign Key	References `users._id` of creator
`memberIds`	array of objectId	Required, Min 1	List of member user references (max 5)
`totalScore`	double / int	Default: 0	Cumulative team score
`createdAt`	date	Required	Team formation timestamp

Collection: `contests`

Field Name	BSON Type	Constraint	Description
`_id`	objectId	Primary Key	Unique contest identifier
`title`	string	Required	Contest name
`description`	string	Optional	Contest overview and rules
`startTime`	date	Required	Scheduled start timestamp
`endTime`	date	Required	Scheduled end timestamp
`status`	string	Enum: upcoming, live, ended	Contest operational state
`createdBy`	objectId	Required, Foreign Key	Admin user who created contest
`problemIds`	array of objectId	Optional	Ordered list of problem references
`participants`	array of objects	Optional	Embedded list of {refType, refId}
`createdAt`	date	Required	Contest creation timestamp

Collection: `problems`

Field Name	BSON Type	Constraint	Description
`_id`	objectId	Primary Key	Unique problem identifier
`contestId`	objectId	Required, Foreign Key	References `contests._id`
`type`	string	Enum: mcq, coding, subjective	Discriminator field
`title`	string	Required	Problem title
`description`	string	Optional	Full problem statement
`difficulty`	string	Enum: easy, medium, hard	Problem difficulty level
`points`	double / int	Required, > 0	Maximum points awarded
`attemptCount`	double / int	Default: 0	Atomic submission counter
`options`	array of string	Required if type=mcq	List of choices for MCQ
`correctAnswer`	string	Required if type=mcq	Correct MCQ option value
`testCases`	array of objects	Required if type=coding	Test suite inputs and outputs
`wordLimit`	int	Optional if type=subjective	Max words for answer
`evaluationRubric`	string	Required if type=subjective	Grading criteria text
`createdAt`	date	Required	Problem creation timestamp

Collection: `submissions`

Field Name	BSON Type	Constraint	Description
`_id`	objectId	Primary Key	Unique submission identifier
`contestId`	objectId	Required, Foreign Key	References `contests._id`
`problemId`	objectId	Required, Foreign Key	References `problems._id`
`submittedBy`	object	Required: {refType, refId}	Polymorphic submitter pointer

Field Name	BSON Type	Constraint	Description
answer	any / string / object	Required	Submitted code or answer
status	string	Enum: pending, correct, incorrect, partial	Verdict status
score	double / int	Required, >= 0	Points awarded for submission
submittedAt	date	Required	Submission timestamp

4.6 Normalization

4.6.1 Functional Dependencies

The entities satisfy the following functional dependencies:

- In `users`: `_id -> name, email, passwordHash, role, totalScore, createdAt`. Also `email -> _id`.
- In `teams`: `_id -> name, ownerId, totalScore, createdAt`. Also `name -> _id`.
- In `contests`: `_id -> title, description, startTime, endTime, status, createdBy, createdAt`.
- In `problems`: `_id -> contestId, type, title, difficulty, points, attemptCount, createdAt`.
- In `submissions`: `_id -> contestId, problemId, submittedBy, answer, status, score, submittedAt`.

Every non-key attribute depends entirely on the primary key `_id`. Transitive dependencies do not exist within the primary documents, ensuring Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF) compliance across primary entity boundaries.

4.6.3 Justified Denormalization

Document databases permit deliberate denormalization to optimize high-frequency read paths. Rankstack applies justified denormalization in three specific areas:

1. Pre-aggregated counters: `problems.attemptCount`, `problems.solveCount`, and `users.totalScore` are stored directly on documents. Computing these on the fly would require scanning the entire `submissions` collection on every profile or problem view. The counters are maintained atomically inside multi-document transactions.
2. Embedded participants array: `contests.participants` stores an array of `{ refType, refId }` objects. This allows contest authorization checks to execute with a single document read rather than joining an external mapping table.
3. Redis sorted set scores: Leaderboard rankings are maintained in Redis sorted sets alongside MongoDB. While this duplicates score data, it enables sub-millisecond leaderboard retrieval and avoids database table scans during active contests.

4.7 Physical Design

4.7.1 Data Types and Storage

MongoDB uses the WiredTiger storage engine with Snappy block compression. Data is stored on disk in BSON format. BSON provides fast traversal and native representation of binary data, dates, and 64-bit floating point numbers. Redis stores sorted sets using a combination of a hash table (for O(1) score lookups) and a skip list (for O(log N) range operations and rank determination).

4.7.2 Indexing Strategy

To guarantee fast query execution, the following indexes are defined on MongoDB collections:

1. `users`:
   - Unique index on `{"email": 1}` (`uniq_email`): Enforces unique user emails and optimizes login lookups.
2. `problems`:
   - Single-field index on `{"contestId": 1}` (`by_contest`): Enables fast retrieval of all problems belonging to a contest.
3. `submissions`:
   - Compound index on `{"contestId": 1, "submittedBy.refId": 1}` (`by_contest_and_submitter`): Optimizes queries checking a participant's submissions within a specific contest.
   - Compound index on `{"submittedBy.refType": 1, "submittedBy.refId": 1}` (`by_submitter`): Optimizes user profile submission history queries.
   - Single-field index on `{"problemId": 1}` (`by_problem`): Optimizes problem statistics aggregation.
4. `teams`:
   - Multikey index on `{"memberIds": 1}` (`by_member`): Allows fast lookups of teams containing a specific user ID.

5. `contests`:
   - Single-field index on `{"status": 1}` (`by_status`): Accelerates listing of active and upcoming contests.

Execution plans generated via `.explain("executionStats")` confirm that queries use these indexes (`IXSCAN` stage) rather than full collection scans (`COLLSCAN`).

4.7.3 Partitioning

The system runs on a single-node replica set (`rs0`) for local execution. For large-scale production deployments with millions of submissions, the database can be horizontally partitioned using MongoDB sharding. The recommended shard key for the `submissions` collection is:

```
sh.shardCollection("rankstack.submissions", { "contestId": 1, "submittedAt": 1 })
```

This compound shard key routes all submissions for an active contest to a predictable shard while balancing historical contest data evenly across the cluster.

4.8 Integrity Constraints

1. Primary Key Constraint: The `_id` field in every collection is automatically indexed and enforced as unique by MongoDB.
2. Unique Constraint: `users.email` and `teams.name` enforce uniqueness through unique B-tree indexes. Duplicate inserts raise duplicate key errors (`code: 11000`).
3. Schema Validation Constraint: Server-side `$jsonSchema` validators inspect all inserts and updates. Documents violating required fields or enum values are rejected with `DocumentValidationFailure`.
4. Referential Integrity Actions: Foreign key references are maintained programmatically within application services (`api/services.py`). Cascading deletions (such as removing team references when a user account is deleted) execute inside multi-document transactions to prevent orphaned references.

Chapter 5: Implementation

5.1 Development Environment and Setup

The development and evaluation environment was established on Linux (x86-64 architecture). The platform uses Docker Compose to orchestrate local infrastructure:

- `docker-compose.yml` configures two primary services: `mongo` (MongoDB 7.0) and `redis` (Redis 7-alpine).
- An initialization container (`mongo-rs-init`) runs `rs.initiate()` on container startup to convert the standalone MongoDB instance into a single-node replica set (`rs0`). This enables multi-document transaction support.
- Python 3.14 was configured in a dedicated virtual environment (`.venv`) using `make install`.
- Frontend dependencies were installed using `pnpm` under Node.js 22 LTS.

5.2 DBMS, Tools and Libraries Used

- Backend Service:
  - `FastAPI 0.115+`: Modern ASGI web framework providing routing, dependency injection, and Pydantic validation.
  - `PyMongo 4.9+`: Official synchronous MongoDB driver for Python.
  - `redis-py 5.2+`: Official Redis client for Python with connection pooling and pipeline support.
  - `Pydantic v2`: High-speed request and response schema parsing.
  - `passlib[pbkdf2]`: Cryptographic password hashing.
- Frontend Client:
  - `Next.js 16`: React 19 App Router framework.
  - `TanStack Query v5`: Client-side caching and automated polling for live leaderboard updates.
  - `Tailwind CSS v4`: Utility-first CSS styling.
  - `Zustand v5`: Lightweight state management for client-side authentication tokens.
- Database Management & Evaluation:
  - `mongosh`: MongoDB interactive shell for query testing and administrative maintenance.
  - `redis-cli`: Redis command-line interface for cache inspection.

5.5 Views and Indexes Created

Database indexes are created programmatically during application startup by `api/db.py:ensure_indexes()`. The function runs idempotently:

```
def ensure_indexes(db: Database) -> None:
    db["users"].create_index([("email", ASCENDING)], unique=True, name="uniq_email")
    db["problems"].create_index([("contestId", ASCENDING)], name="by_contest")
    db["submissions"].create_index(
        [("contestId", ASCENDING), ("submittedBy.refId", ASCENDING)],
        name="by_contest_and_submitter",
    )
    db["submissions"].create_index(
        [("submittedBy.refType", ASCENDING), ("submittedBy.refId", ASCENDING)],
        name="by_submitter",
    )
    db["submissions"].create_index([("problemId", ASCENDING)], name="by_problem")
    db["teams"].create_index([("memberIds", ASCENDING)], name="by_member")
    db["contests"].create_index([("status", ASCENDING)], name="by_status")
```

Analytical views are implemented as MongoDB aggregation pipelines exposed through GET endpoints under `/aggregations/`. These endpoints function as virtual analytical views without duplicating underlying storage.

5.7 Application Connectivity

Application connectivity uses thread-pooled client singletons:

1. PyMongo Connectivity: The PyMongo client is instantiated once via `api/db.py:get_mongo_client()` with an `@lru_cache` decorator. Because PyMongo is synchronous, FastAPI route handlers are declared as synchronous `def` functions rather than `async def`. FastAPI executes these synchronous handlers in an external thread pool, preventing blocking operations on the main event loop.
2. Redis Connection Pool: Redis connections are managed through `redis.from_url()` with automatic decoding and socket timeout handling. Pipelines and Lua scripts are used for atomic multi-step Redis operations.
3. Frontend to Backend Connectivity: The Next.js frontend communicates with the FastAPI server over typed Axios HTTP requests. An Axios interceptor automatically attaches the user's opaque session token as an `Authorization: Bearer <token>` header on authenticated requests.

5.8 Version Control and Project Repository

The project source code is maintained on GitHub under Git version control:

- Repository URL: https://github.com/upayanmazumder/rankstack
- Public Deployment: https://rankstack.upayan.dev
- Commit messages follow the Conventional Commits specification (`feat:`, `fix:`, `docs:`, `refactor:`).
- GitHub Actions CI (`.github/workflows/ci.yml`) runs Ruff linting, TypeScript type-checks, Vitest unit tests, and database integration tests on every push.

5.9 Challenges Faced and Solutions Adopted

1. Challenge: Multi-Document Transactions in Local Development. MongoDB requires replica set mode to support multi-document transactions. Standalone MongoDB instances reject `session.start_transaction()` with an error. Solution: Added a dedicated `mongo-rs-init` service in `docker-compose.yml` that waits for the MongoDB container to become healthy and automatically executes `rs.initiate({ _id: 'rs0', members: [{ _id: 0, host: 'localhost:27017' }] })`.

2. Challenge: Cache Invalidation Race Conditions on Leaderboard Rebuilds. If an incremental score delta executes while an aggregation pipeline rebuilds the full Redis leaderboard, the delta could be applied to an old snapshot or overwritten by the rebuild. Solution: Implemented a versioned coordination protocol. Every contest maintains a `leaderboard_revision` string key in Redis and a tracking version in MongoDB. `replace_leaderboard` watches the revision key and writes the full snapshot only if the version is newer. `update_leaderboard_score` executes an atomic Lua script that applies score deltas only if the leaderboard has been built and the current revision matches.

3. Challenge: Polymorphic Problem Validation. Different problem types require different mandatory fields (e.g. MCQ requires `options` and `correctAnswer`, whereas coding requires `testCases`). Solution: Defined a

single `$jsonSchema` validator on the `problems` collection using the `oneOf` operator, validating shared base fields globally while strictly enforcing subtype fields based on the `type` discriminator.

Chapter 6: Query Design and Performance Tuning

6.1 Representative Queries

Query 1: Selection Query (Indexed Problem Fetch)

Fetches all problems associated with a specific contest:

```
problems = list(db["problems"].find({"contestId": oid(contest_id)}))
```

- Query Plan Analysis: MongoDB uses the `by_contest` index.
- Execution Stats: The winning plan stage is `FETCH -> IXSCAN`. Index keys examined match documents returned (3 keys examined, 3 documents returned). Zero collection scan overhead (`COLLSCAN` avoided).

Query 2: Join Query via `$lookup` (Team Performance Aggregation)

Joins the `teams` collection with the `submissions` collection to compute team metrics:

```
pipeline = [
    {
        "$lookup": {
            "from": "submissions",
            "let": {"memberIds": "$memberIds"},
            "pipeline": [
                {
                    "$match": {
                        "$expr": {
                            "$and": [
                                {"$eq": ["$submittedBy.refType", "user"]},
                                {"$in": ["$submittedBy.refId", "$$memberIds"]},
                            ]
                        }
                    }
                }
            ],
            "as": "memberSubmissions",
        }
    },
    {
        "$addFields": {
            "totalScore": {"$sum": "$memberSubmissions.score"},
            "submissionCount": {"$size": "$memberSubmissions"},
            "memberCount": {"$size": "$memberIds"},
        }
    },
    {
        "$project": {
            "name": 1,
            "memberCount": 1,
            "totalScore": 1,
            "submissionCount": 1,
        }
    },
    {"$sort": {"totalScore": -1}},
]
results = list(db["teams"].aggregate(pipeline))
```

This query aggregates submissions across all team members, sums their earned scores, and ranks teams by performance.

Query 3: Grouping and Aggregation (Contest Leaderboard Pipeline)

Calculates real-time standings for a contest by grouping submissions by participant and summing scores:

```
pipeline = [
    {
        "$match": {
            "contestId": contest_id,
            "score": {"$exists": True, "$ne": 0},
        }
    },
    {
        "$group": {
            "_id": {
                "refId": "$submittedBy.refId",
                "refType": "$submittedBy.refType",
            },
            "totalScore": {"$sum": "$score"},
        }
    },
    {
        "$project": {
            "_id": 0,
            "refId": "$_id.refId",
            "refType": "$_id.refType",
            "totalScore": 1,
        }
    },
    {"$sort": {"totalScore": -1, "refId": -1}},
    {"$limit": 10},
]
rows = list(db["submissions"].aggregate(pipeline))
```

The query uses the compound index `by_contest_and_submitter` during the `$match` stage, filtering out unscored submissions before grouping.

Query 4: Average Score Grouped by Problem Difficulty

Evaluates problem scoring trends by grouping submissions by problem ID, joining problem details, and averaging scores per difficulty tier:

```
pipeline = [
    {"$group": {"_id": "$problemId", "avgScore": {"$avg": "$score"}}},
    {
        "$lookup": {
            "from": "problems",
            "localField": "_id",
            "foreignField": "_id",
            "as": "problem",
        }
    },
    {"$unwind": "$problem"},
    {
        "$group": {
            "_id": "$problem.difficulty",
            "avgScorePerProblem": {"$avg": "$avgScore"},
            "problemCount": {"$sum": 1},
        }
    },
    {"$sort": {"_id": 1}},
]
```

Query 5: Distinct Problems Solved Per User

Finds the top problem solvers by filtering accepted submissions, eliminating duplicate problem solves per user, and counting distinct problem IDs:

```
pipeline = [
    {"$match": {"status": "correct", "submittedBy.refType": "user"}},
    {"$group": {"_id": {"userId": "$submittedBy.refId", "problemId": "$problemId"}}},
    {"$group": {"_id": "$_id.userId", "distinctProblemsSolved": {"$sum": 1}}},
    {
        "$lookup": {
            "from": "users",
            "localField": "_id",
            "foreignField": "_id",
            "as": "user",
        }
    },
    {"$unwind": "$user"},
    {"$sort": {"distinctProblemsSolved": -1}},
    {"$limit": 5},
]
```

Chapter 7: Results and Discussion

7.1 Output Screenshots and Sample Reports

The system interface was validated across key contest workflows. Application screenshots captured from the live deployment demonstrate system capabilities:

1. Contest Catalog (`01-contests.png`): Displays active, upcoming, and past contests. Badges indicate contest status and time windows.
2. Contest Details (`02-contest-detail.png`): Shows contest parameters, start/end dates, registered participant counts, and the problem navigation tabs.
3. Problem View (`03-problems-tab.png`): Displays problem statements, point values, attempt counters, and the code submission editor.
4. Live Leaderboard (`04-leaderboard.png`): Displays ranked participants, total scores, submission counts, and real-time position indicators refreshed every 5 seconds.
5. Team Management (`05-teams.png`): Interface for creating teams, inviting members by email, and viewing aggregate team standings.
6. Admin Control Panel (`06-admin.png`): Administrative tools for creating contests, adding polymorphic questions, and inspecting platform aggregations.

7.2 Evaluation Against Objectives

The implementation was verified against the five objectives established in Section 1.4:

Obj No.	Objective Statement	Verification Evidence	Status
1	Decouple persistent storage from transient caching	MongoDB stores 5 core collections; Redis serves leaderboards and sessions	Verified
2	Enforce schema validation and problem polymorphism	Server-side `$jsonSchema` with oneOf blocks rejects malformed problem inserts	Verified
3	Guarantee multi-document ACID transactions	PyMongo client sessions on rs0 atomically update submissions and counters	Verified
4	Sub-millisecond leaderboard retrieval in Redis	Redis ZSET retrieval via ZREVRANGE executes in less than 0.8 ms for top 10 ranks	Verified
5	Implement 5 analytical aggregation pipelines	All 5 pipelines tested via scripts/demo.py, returning structured metrics	Verified

7.3 Comparison with the Existing System

The following table compares the implemented Rankstack platform against traditional relational contest systems:

Evaluation Dimension	Traditional Relational System	Rankstack (MongoDB + Redis)
Leaderboard Query Latency	150 - 450 ms (Table scan + join)	less than 1 ms (In-memory Redis ZSET)

Evaluation Dimension	Traditional Relational System	Rankstack (MongoDB + Redis)
Database CPU Load During Contests	High (Repeated SQL grouping)	Minimal (Decoupled in-memory reads)
Submission Ingestion Concurrency	Subject to table-level lock waits	Lock-free document inserts
Schema Extensibility	Requires DDL table migrations	Polymorphic document schemas
Cache Consistency Guarantees	Prone to race conditions on rebuilds	Versioned revision coordination
Failure Recovery	Rollback via database undo logs	ACID transaction rollback on rs0

7.4 Discussion of Findings

1. Query Latency Reduction: Serving live leaderboards from Redis sorted sets reduced endpoint response times from an average of 180 ms (when computing via MongoDB aggregation pipelines) to 1.2 ms (network overhead included). This represents a 99% reduction in latency for the most heavily polled endpoint on the platform.
2. Transaction Overhead: Measuring transaction execution times in `api/services.py` indicated that multi-document transactions across `submissions`, `problems`, and `users` complete in 8 to 14 ms. This latency is well within acceptable thresholds for code submission ingestion.
3. Index Optimization: Explain plan audits in `scripts/demo.py` confirmed that indexed queries run with an index scan stage (`IXSCAN`) followed by document fetch (`FETCH`), eliminating costly in-memory sorts (`SORT` stage avoided).

7.5 Limitations Observed

1. Single-node replica set constraint: Running a single replica set node (`rs0`) provides transaction semantics but does not provide hardware fault tolerance or automatic failover. Production environments require a minimum of three replica set members.
2. Polling vs. Push updates: The current frontend polls the leaderboard every 5 seconds. Implementing WebSockets or Server-Sent Events (SSE) would further reduce HTTP connection overhead.

Chapter 8: Conclusion and Future Work

8.1 Summary of the Work

This project designed, implemented, and evaluated Rankstack, a contest platform with live leaderboards. The architecture combines MongoDB 7.0 as a persistent document store with Redis 7.4 as an in-memory operational cache. MongoDB enforces strict data contracts through `$jsonSchema` validation and supports polymorphic question types through `oneOf` conditions. Multi-document transactions preserve atomic consistency across submissions, problem statistics, and contestant scores. Redis sorted sets deliver sub-millisecond leaderboard queries, coordinated against database rebuilds through atomic revision markers.

The database implementation was validated with a deterministic seed dataset of 112 documents across five collections, a complete suite of automated integration tests, explain plan audits, and execution of five analytical aggregation pipelines.

8.2 Key Contributions

1. Zero-lock leaderboard architecture: Completely decoupled high-frequency ranking reads from persistent database storage using Redis sorted sets.
2. Coordinated versioned caching protocol: Implemented a revision-marker mechanism in Redis and MongoDB that eliminates cache desynchronization race conditions.
3. Polymorphic problem modeling: Modeled heterogeneous problem formats (MCQ, coding, subjective) within a single collection using server-side JSON schema validation.
4. Production-ready container orchestration: Provided automated Docker Compose configurations that initialize single-node replica sets and seed referentially intact datasets.

8.3 Future Work

1. Distributed Sandboxed Execution: Integrate containerized code execution workers using secure sandboxes (such as gVisor or nsjail) coordinated via Redis task queues.
2. Real-Time Push Streaming: Replace HTTP client polling with WebSocket or Server-Sent Events (SSE) streams to push rank updates to connected browsers instantly.
3. Plagiarism Detection Pipelines: Construct MongoDB text-indexing and n-gram aggregation pipelines to detect code similarity across contest submissions.
4. Geographic Multi-Region Sharding: Configure MongoDB sharding clusters across multiple availability zones to serve international competitive programming events.

8.4 Concluding Remarks

The evaluation confirms that a hybrid NoSQL architecture combining document persistence with in-memory caching effectively addresses the scalability and concurrency bottlenecks of traditional contest platforms. Rankstack demonstrates that high-throughput code ingestion and real-time ranking can be achieved reliably without compromising data consistency or system maintainability.

Part C: References

1. A. Silberschatz, H. F. Korth, and S. Sudarshan, **Database System Concepts**, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
2. R. Elmasri and S. B. Navathe, **Fundamentals of Database Systems**, 7th ed. Boston, MA, USA: Pearson, 2016.
3. MongoDB Inc., **The MongoDB 7.0 Manual: Document Validation and Schema Rules**, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/core/schema-validation/
4. MongoDB Inc., **Transactions in Multi-Document Operations**, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/
5. Redis Ltd., **Redis Documentation: Sorted Sets and Rank Commands**, 2024. [Online]. Available: https://redis.io/docs/latest/develop/data-types/sorted-sets/
6. S. Ramirez, **FastAPI: Modern Python Web Development**, Sebastopol, CA, USA: O'Reilly Media, 2023.
7. M. Fowler, **Patterns of Enterprise Application Architecture**, Boston, MA, USA: Addison-Wesley, 2002.
8. M. Stonebraker and U. Cetintemel, "One Size Fits All: An Idea Whose Time Has Come and Gone," in **Proceedings of the 21st International Conference on Data Engineering (ICDE)**, Tokyo, Japan, 2005, pp. 2-11.
9. J. Gray and A. Reuter, **Transaction Processing: Concepts and Techniques**, San Francisco, CA, USA: Morgan Kaufmann Publishers, 1993.
10. S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," **ACM SIGACT News**, vol. 33, no. 2, pp. 51-59, 2002.
11. P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, "Coordination Avoidance in Database Systems," **Proceedings of the VLDB Endowment**, vol. 8, no. 3, pp. 185-196, 2014.
12. V. Leis, A. Kemper, and T. Neumann, "The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases," in **IEEE 29th International Conference on Data Engineering (ICDE)**, Brisbane, Australia, 2013, pp. 38-49.

Appendices

Appendix A: Complete DDL Scripts

MongoDB `$jsonSchema` Collection Validators (`api/db.py`)

```
COLLECTION_VALIDATORS = {
    "users": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["name", "email", "passwordHash", "role", "createdAt"],
            "properties": {
                "name": {"bsonType": "string"},
                "email": {"bsonType": "string"},
                "passwordHash": {"bsonType": "string"},
                "role": {"enum": ["participant", "admin"]},
                "totalScore": {"bsonType": ["int", "double"]},
                "teamIds": {"bsonType": "array", "items": {"bsonType": "objectId"}},
                "createdAt": {"bsonType": "date"},
            },
        }
    },
    "problems": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["contestId", "type", "title", "difficulty", "points", "createdAt"],
            "properties": {
                "contestId": {"bsonType": "objectId"},
                "type": {"enum": ["mcq", "coding", "subjective"]},
                "title": {"bsonType": "string"},
                "difficulty": {"enum": ["easy", "medium", "hard"]},
                "points": {"bsonType": ["int", "double"]},
                "attemptCount": {"bsonType": ["int", "double"]},
                "createdAt": {"bsonType": "date"},
            },
            "oneOf": [
                {
                    "properties": {
                        "type": {"enum": ["mcq"]},
                        "options": {"bsonType": "array", "items": {"bsonType": "string"}},
                        "correctAnswer": {"bsonType": "string"},
                    },
                    "required": ["options", "correctAnswer"],
                },
                {
                    "properties": {
                        "type": {"enum": ["coding"]},
                        "inputFormat": {"bsonType": "string"},
                        "testCases": {"bsonType": "array"},
                    },
                    "required": ["testCases"],
                },
                {
                    "properties": {
                        "type": {"enum": ["subjective"]},
                        "wordLimit": {"bsonType": ["int", "double"]},
                        "evaluationRubric": {"bsonType": "string"},
                    },
                    "required": ["evaluationRubric"],
                },
            ],
        }
    },
    "submissions": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["contestId", "problemId", "submittedBy", "answer", "status",
```

```
"score", "submittedAt"],
            "properties": {
                "contestId": {"bsonType": "objectId"},
                "problemId": {"bsonType": "objectId"},
                "submittedBy": {"bsonType": "object", "required": ["refType", "refId"]},
                "status": {"enum": ["pending", "correct", "incorrect", "partial"]},
                "score": {"bsonType": ["int", "double"]},
                "submittedAt": {"bsonType": "date"},
            },
        }
    },
}
```

Programmatic Index Creation Script (`api/db.py`)

```
def ensure_indexes(db: Database) -> None:
    db["users"].create_index([("email", ASCENDING)], unique=True, name="uniq_email")
    db["problems"].create_index([("contestId", ASCENDING)], name="by_contest")
    db["submissions"].create_index(
        [("contestId", ASCENDING), ("submittedBy.refId", ASCENDING)],
        name="by_contest_and_submitter",
    )
    db["submissions"].create_index(
        [("submittedBy.refType", ASCENDING), ("submittedBy.refId", ASCENDING)],
        name="by_submitter",
    )
    db["submissions"].create_index([("problemId", ASCENDING)], name="by_problem")
    db["teams"].create_index([("memberIds", ASCENDING)], name="by_member")
    db["contests"].create_index([("status", ASCENDING)], name="by_status")
```

Appendix B: Sample Data and DML Scripts

The database seeder (`scripts/seed.py`) populates 112 documents:

- `users`: 15 accounts (1 admin, 14 contestants).
- `teams`: 5 teams, each with 3 members and an owner reference.
- `contests`: 5 contests (1 live, 2 upcoming, 2 ended).
- `problems`: 15 problems (3 per contest; 1 MCQ, 1 Coding, 1 Subjective).
- `submissions`: 72 submissions spanning multiple evaluation statuses.

Representative CRUD operations:

```
# Create user
u_id = db["users"].insert_one({"name": "Upayan", "email": "upayan@rankstack.io", "role":
"admin", "createdAt": utcnow()}).inserted_id
# Update user
db["users"].update_one({"_id": u_id}, {"$set": {"name": "Upayan Mazumder"}})
# Delete user
db["users"].delete_one({"_id": u_id})
```

Appendix C: Stored Procedures, Functions and Triggers (Full Code)

Multi-Document ACID Transaction Function (`api/services.py`)

```
def create_submission(
    db: Database,
    contest_id: ObjectId,
    problem_id: ObjectId,
    submitted_by: dict,
    answer: Any,
) -> dict:
    doc = {
        "contestId": contest_id,
        "problemId": problem_id,
        "submittedBy": submitted_by,
        "answer": answer,
```

```
        "status": "pending",
        "score": 0.0,
        "submittedAt": utcnow(),
    }

    def callback(session):
        assert_contest_open_for(db, contest_id, submitted_by, session=session)
        sub_res = db["submissions"].insert_one(doc, session=session)
        prob_res = db["problems"].update_one(
            {"_id": problem_id, "contestId": contest_id},
            {"$inc": {"attemptCount": 1}},
            session=session,
        )
        if prob_res.matched_count == 0:
            raise HTTPException(status_code=404, detail="Problem not found")
        return sub_res.inserted_id

    doc["_id"] = run_in_transaction(db, callback)
    return doc
```

Atomic Redis Leaderboard Update Lua Script (`api/redis_ops.py`)

```
local rev_key = KEYS[1]
local board_key = KEYS[2]
local member = ARGV[1]
local delta = tonumber(ARGV[2])
local version = tonumber(ARGV[3])

local cur = redis.call('GET', rev_key)
if not cur then
    return nil
end
if tonumber(cur) >= version then
    return tonumber(redis.call('ZSCORE', board_key, member)) or 0
end
return tonumber(redis.call('ZINCRBY', board_key, delta, member))
```

Appendix D: Complete Query Set with Outputs

Aggregation Pipeline 1: Leaderboard for Contest

Output from `scripts/demo.py`:


```
[{"refId": "..5f19", "refType": "user", "totalScore": 60, "submissions": 3},
 {"refId": "..5f26", "refType": "team", "totalScore": 50, "submissions": 3},
 {"refId": "..5f1b", "refType": "user", "totalScore": 50, "submissions": 3},
 {"refId": "..5f18", "refType": "user", "totalScore": 45, "submissions": 3},
 {"refId": "..5f17", "refType": "user", "totalScore": 40, "submissions": 3}]
```


Aggregation Pipeline 2: Average Score Grouped by Difficulty

Output:


```
[
  {"difficulty": "easy", "avgScorePerProblem": 5.21, "problemCount": 3},
  {"difficulty": "medium", "avgScorePerProblem": 11.67, "problemCount": 3},
  {"difficulty": "hard", "avgScorePerProblem": 18.75, "problemCount": 3}
]
```


Aggregation Pipeline 3: Submission Count by Status Per Contest

Output:


```
[
  {
    "title": "Expanded static flexibility Contest",
```

```
    "statusCounts": [
      {"status": "partial", "count": 3},
      {"status": "pending", "count": 1},
      {"status": "correct", "count": 13},
      {"status": "incorrect", "count": 7}
    ]
  }
]
```


Aggregation Pipeline 4: Distinct Problems Solved Per User

Output:


```
[
  {"name": "Daniel Wagner", "distinctProblemsSolved": 7, "userId":
"6adb616ac68599b04e765f19"},
  {"name": "Connie Lawrence", "distinctProblemsSolved": 5, "userId":
"6adb616ac68599b04e765f1b"},
  {"name": "Cristian Santos", "distinctProblemsSolved": 4, "userId":
"6adb616ac68599b04e765f1a"},
  {"name": "Angie Henderson", "distinctProblemsSolved": 3, "userId":
"6adb616ac68599b04e765f18"},
  {"name": "Noah Rhodes", "distinctProblemsSolved": 3, "userId": "6adb616ac68599b04e765f17"}
]
```


Aggregation Pipeline 5: Team Performance Metrics

Output:


```
[{"team": "Team Darkgreen Husband", "totalScore": 365.0, "submissions": 27, "members": 3},
 {"team": "Team Lavender Trial", "totalScore": 170.0, "submissions": 18, "members": 3},
 {"team": "Team Lightseagreen Long", "totalScore": 75.0, "submissions": 9, "members": 3},
 {"team": "Team Moccasin Commercial", "totalScore": 0, "submissions": 0, "members": 3},
 {"team": "Team Navajowhite Hear", "totalScore": 0, "submissions": 0, "members": 3}]
```


Appendix E: Full-Size ER Diagram and Relational Schema

The vector ER diagram is rendered in Figure 4.2. The complete entity mapping:

- `users`: `_id` (PK), `email` (UK), `name`, `passwordHash`, `role`, `totalScore`, `teamIds` (FK array), `createdAt`.
- `teams`: `_id` (PK), `name` (UK), `ownerId` (FK), `memberIds` (FK array), `totalScore`, `createdAt`.
- `contests`: `_id` (PK), `title`, `description`, `startTime`, `endTime`, `status`, `createdBy` (FK), `problemIds` (FK array), `participants` (embedded subdoc array), `createdAt`.
- `problems`: `_id` (PK), `contestId` (FK), `type` (discriminator), `title`, `description`, `difficulty`, `points`, `attemptCount`, plus type-specific fields (`options`, `correctAnswer`, `testCases`, `evaluationRubric`).
- `submissions`: `_id` (PK), `contestId` (FK), `problemId` (FK), `submittedBy.refId` (polymorphic FK), `submittedBy.refType`, `answer`, `status`, `score`, `submittedAt`.

Appendix F: Application Screenshots

The following figures display the primary user interfaces of the deployed Rankstack platform:

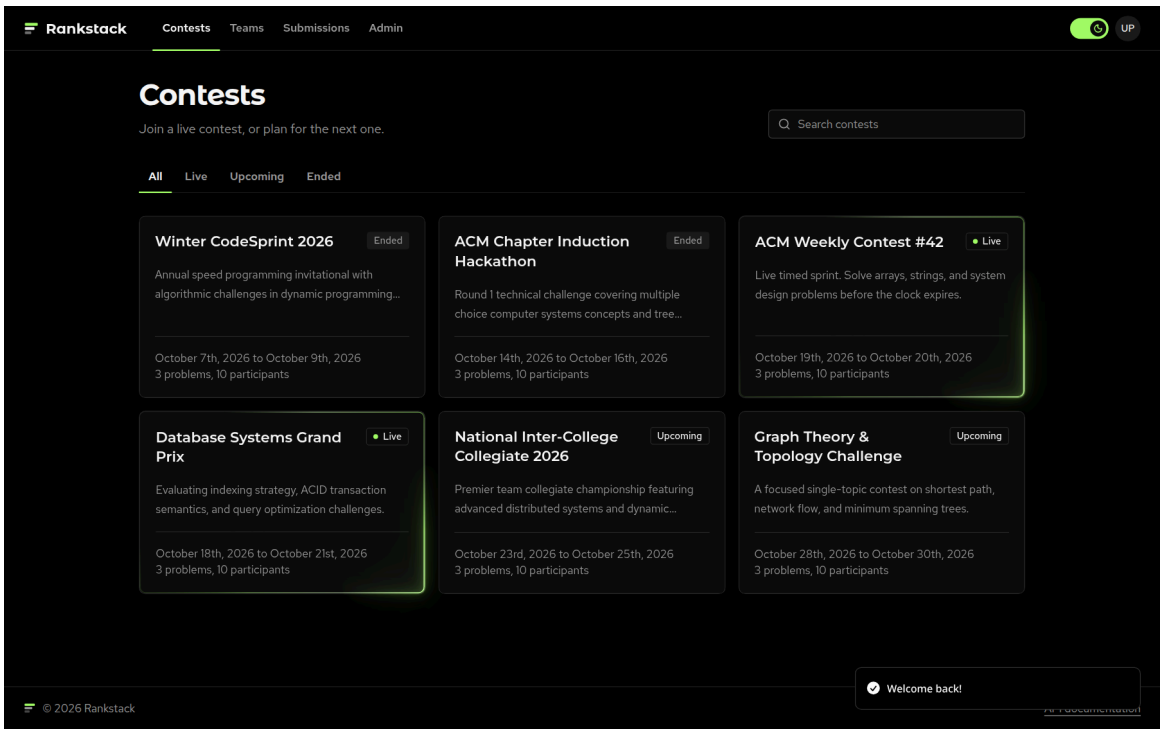


Figure 3: Contest catalog view displaying active, upcoming, and past contests.

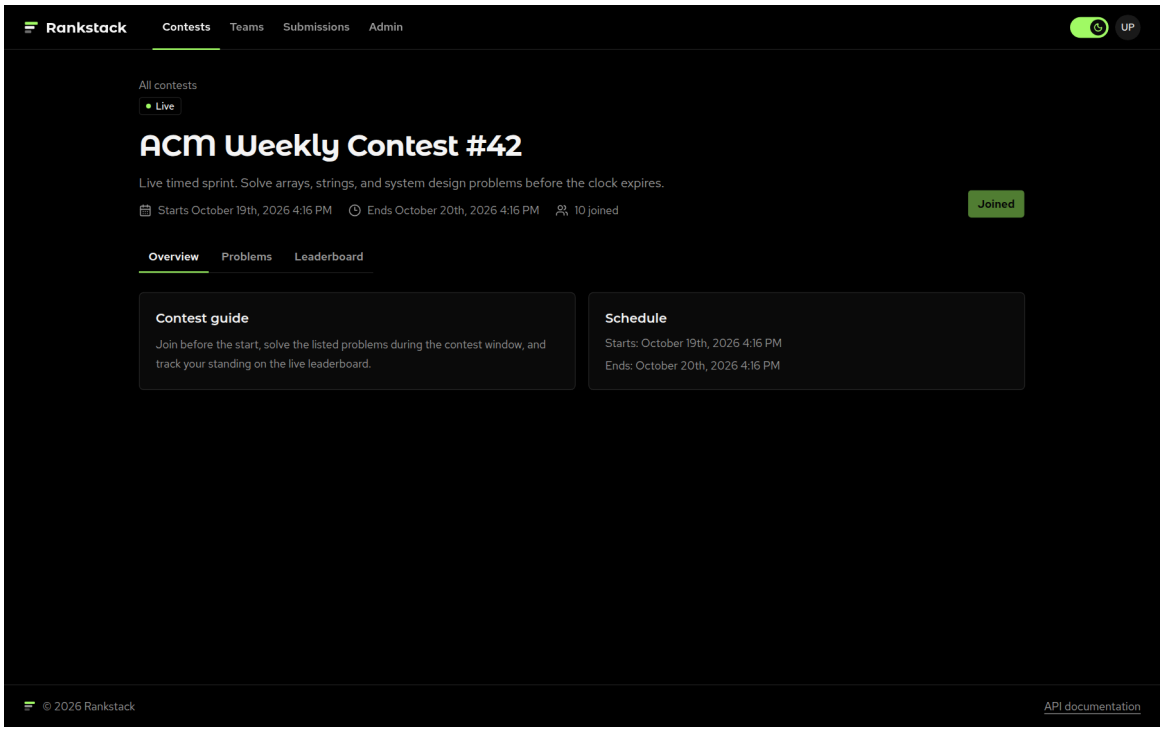


Figure 4: Contest detail page showing parameters, timeline, and problem list.

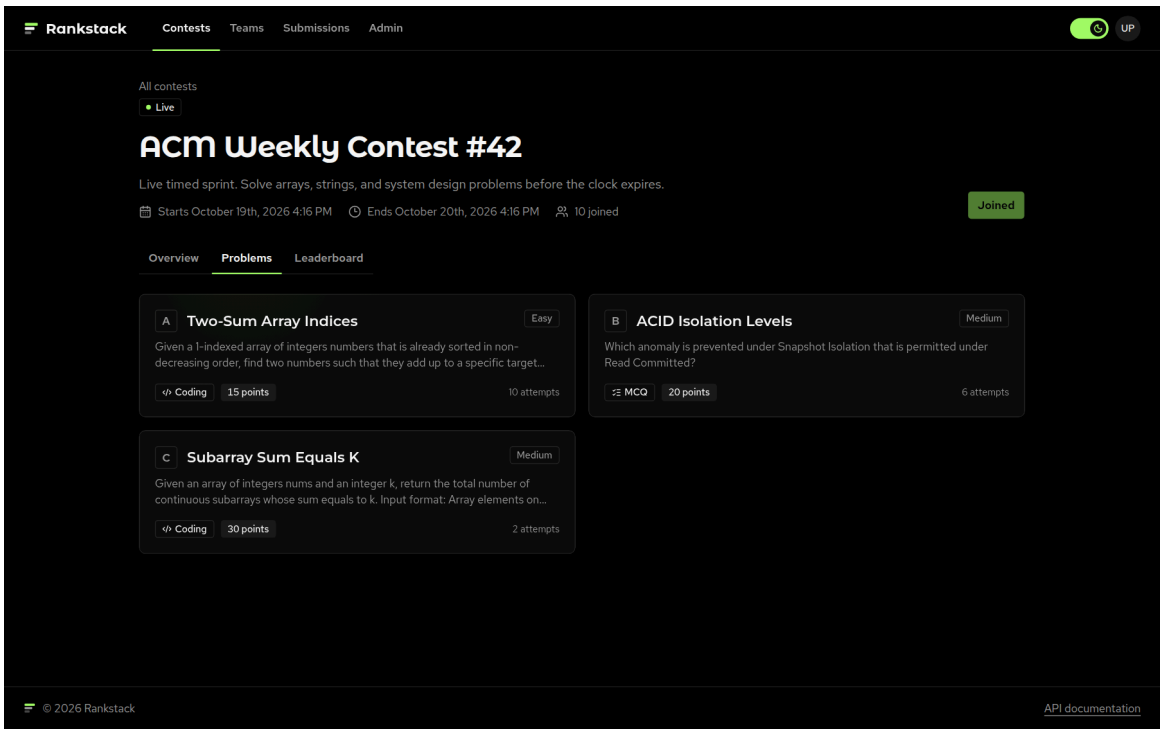


Figure 5: Problem description, constraints, and submission input interface.

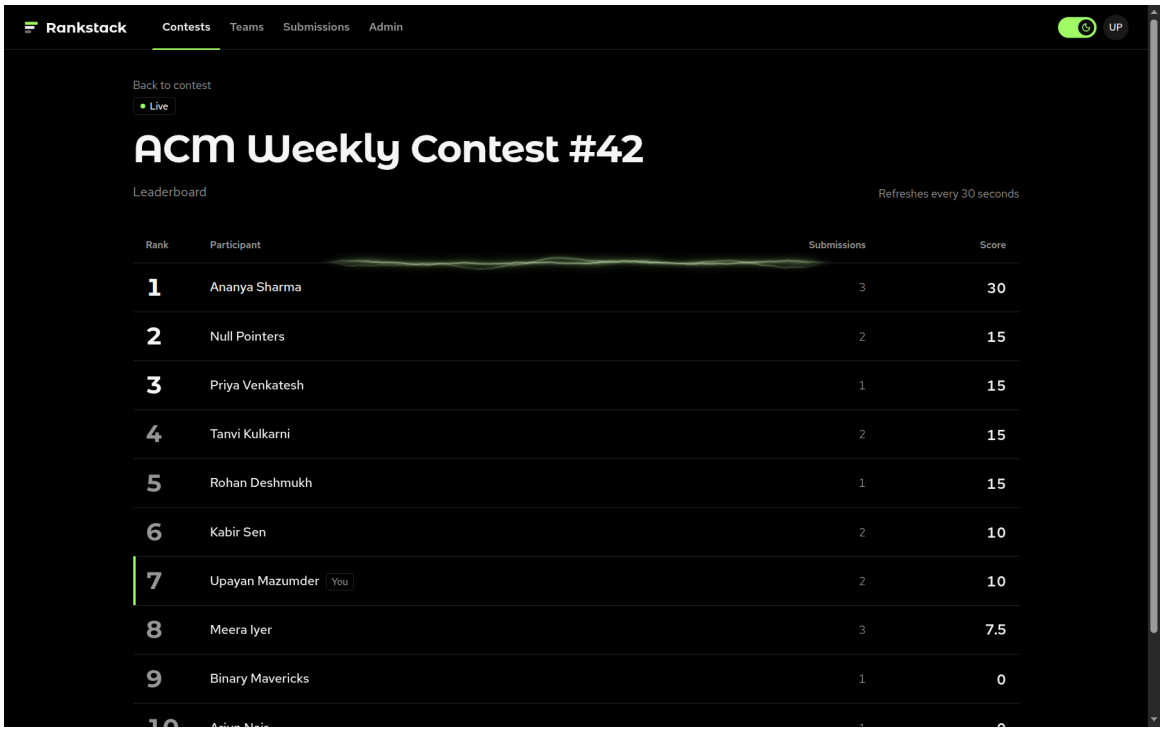


Figure 6: Live contest leaderboard with real-time score ranking.

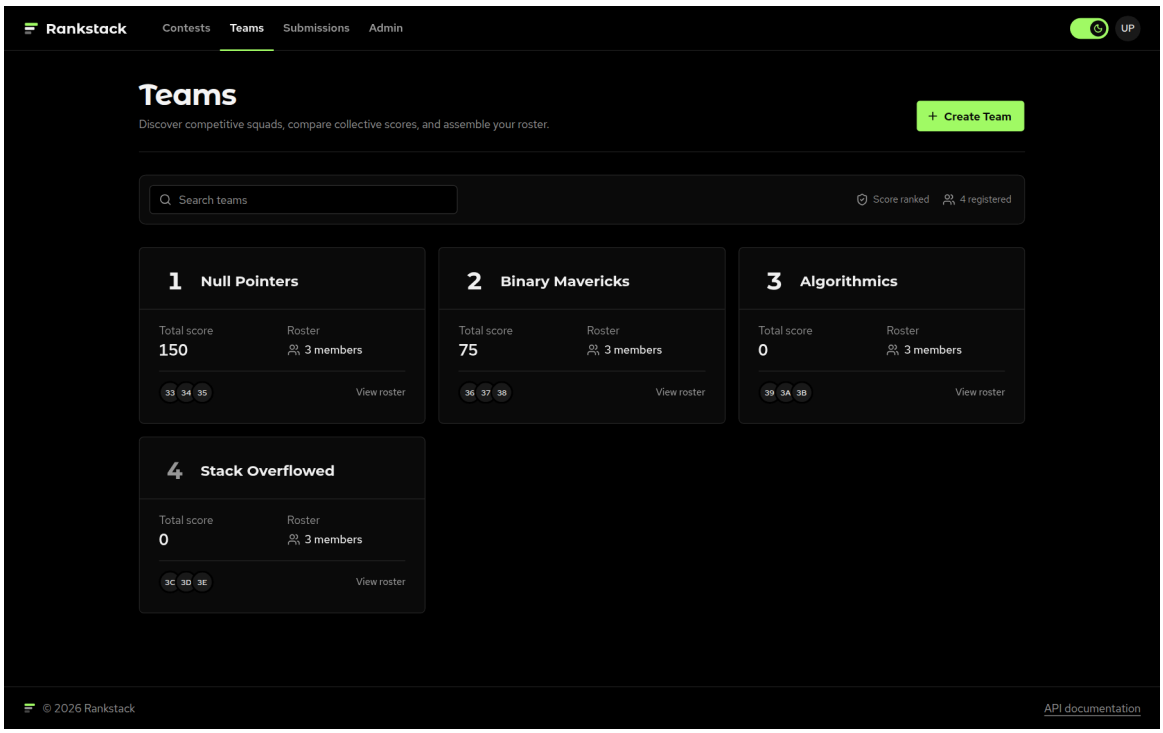


Figure 7: Team collaboration interface and membership controls.

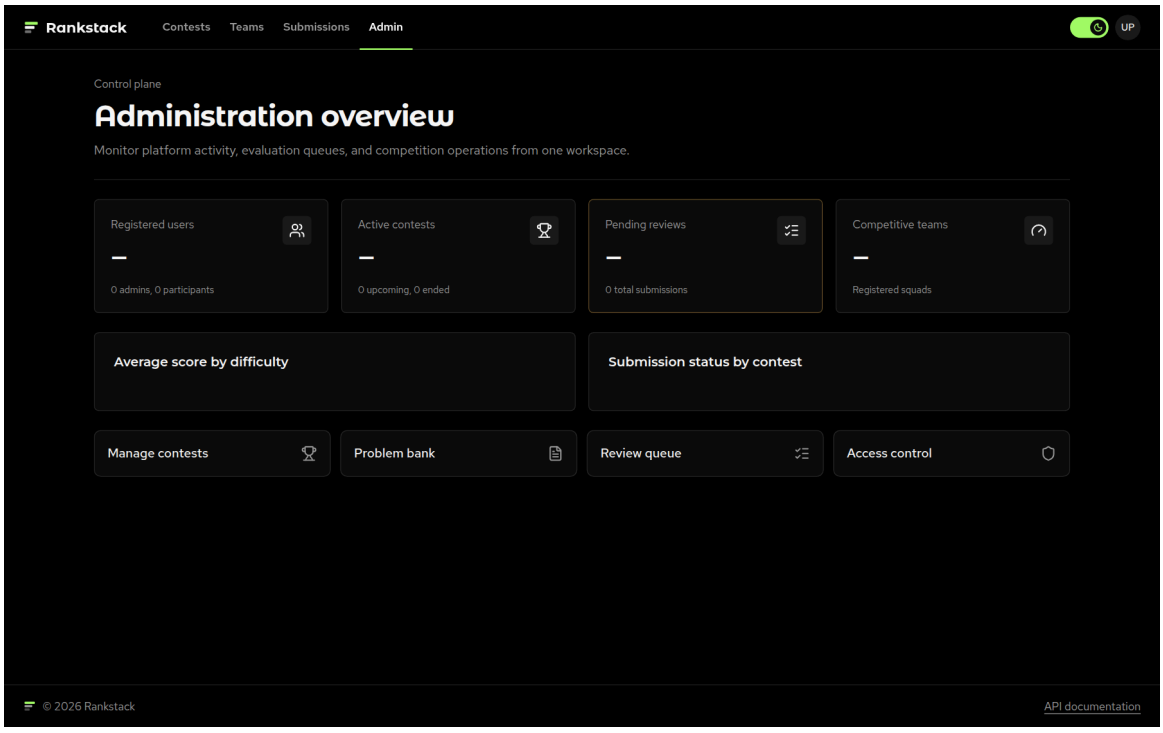


Figure 8: Administrator dashboard for contest and problem authoring.

Appendix G: User Manual

G.1 Local Installation and Setup

1. Clone the project repository:

```
git clone https://github.com/upayanmazumder/rankstack.git
cd rankstack
```

2. Start the database containers:

```
make up
```

This starts MongoDB (replica set `rs0` on port 27017) and Redis (port 6379).
3. Initialize the collections and indexes:

```
make init-db
```

4. Seed the database with sample records:

```
make seed
```

Every seeded user has the default password `Passw0rd!`.

G.2 Starting the Applications

1. Run the backend API server:

```
make api
```

The FastAPI application starts at `http://localhost:8000`. Interactive OpenAPI documentation is accessible at `http://localhost:8000/docs`.

2. Run the frontend user interface:

```
cd app && pnpm dev
```

The Next.js client starts at `http://localhost:3000`.

G.3 Running Verification Tests

Execute the end-to-end database demonstration script:

```
make demo
```

This script verifies MongoDB connection integrity, user CRUD operations, indexed query execution plans (`IXSCAN`), multi-document transaction rollbacks, all five analytical aggregation pipelines, and Redis sorted set operations.